Taller 3: Ordenamiento y Búsqueda

Comparación experimental de algoritmos de ordenamiento y búsqueda

Análisis de Algoritmos
Nombre Apellido (código)
Nombre Apellido (código)
Repositorio: https://github.com/USUARIO/REPOSITORIO
Octubre de 2026

1. Metodología y condiciones del experimento

Implementación. Cada algoritmo se implementó en dos lenguajes con la misma lógica: Java (compilado, con JIT; java version "27.0.1.0.2" 2026-10-21) y Python (interpretado; versión 3.15.1). Las implementaciones se adaptaron de GeeksforGeeks (fuente principal, ver Tabla 1). Los ordenamientos trabajan en sitio sobre el arreglo recibido; las búsquedas devuelven un índice o -1.

Datos. Se generaron una sola vez con una semilla fija (tools/generate_data.py) y se guardaron en archivos de texto plano: arreglos de 1.000, 10.000, 100.000 y 1.000.000 enteros aleatorios de 8 dígitos (entre 10.000.000 y 99.999.999), junto con sus versiones ascendente y descendente y un archivo de 1.000 claves de búsqueda por tamaño (500 presentes y 500 ausentes). Todos los programas leen esos archivos, por lo que todos los algoritmos se evalúan con el mismo conjunto de datos.

Medición. Se mide únicamente la ejecución del algoritmo (la lectura de archivos y la copia del arreglo quedan fuera del tiempo) con System.nanoTime() en Java y time.perf_counter_ns() en Python. Cada medición se repite 5 veces sobre una copia nueva del arreglo; se reporta el promedio y la desviación estándar muestral. En Java se ejecuta antes un calentamiento (10 ejecuciones sobre 10.000 elementos por algoritmo de ordenamiento y 300 pasadas del conjunto de claves por algoritmo de búsqueda) que no entra en el promedio. Tras cada ejecución, y fuera del tiempo medido, se verifica que el arreglo quedó ordenado (y con la misma suma) o que cada búsqueda devolvió el resultado correcto.

Valores estimados y fallos. Antes de ejecutar un tamaño n, el programa estima su tiempo escalando el último promedio medido, $t(n) = t(n_{prev}) \cdot f(n) / f(n_{prev})$, donde f es la complejidad teórica del algoritmo para esa entrada. Si la estimación supera 300 s por ejecución, no se ejecuta y el valor se marca como estimado. Si un algoritmo falla (desbordamiento de pila o límite de recursión), el error queda registrado en el CSV y en la Tabla A.3, y los tamaños mayores se estiman con el mismo escalamiento. En las gráficas los valores estimados se dibujan con línea punteada y marcador hueco (líneas) o barra rayada con prefijo «~» (barras); en las tablas llevan «~» y «(est.)».

Entorno de pruebas. Sistema: macOS-26.6.2-arm64-arm-64bit-Mach-O; Procesador: arm; Python: 3.15.1; Java: java version "27.0.1.0.2" 2026-10-21.

Tabla 1. Fuentes (GeeksforGeeks) usadas para cada algoritmo.

Algoritmo	Fuente
Insertion Sort	https://www.geeksforgeeks.org/insertion-sort-algorithm/
Dual-Pivot QuickSort	https://www.geeksforgeeks.org/dual-pivot-quicksort/
Merge Sort	https://www.geeksforgeeks.org/merge-sort/
Heap Sort	https://www.geeksforgeeks.org/heap-sort/
Radix Sort	https://www.geeksforgeeks.org/radix-sort/
Bucket Sort	https://www.geeksforgeeks.org/bucket-sort-2/
Búsqueda lineal	https://www.geeksforgeeks.org/linear-search/
Búsqueda binaria	https://www.geeksforgeeks.org/binary-search/
Búsqueda ternaria	https://www.geeksforgeeks.org/ternary-search/
Búsqueda por saltos	https://www.geeksforgeeks.org/jump-search/
Búsqueda por interpolación	https://www.geeksforgeeks.org/interpolation-search/
Búsqueda exponencial	https://www.geeksforgeeks.org/exponential-search/

2. Punto 1 - Algoritmos de ordenamiento

Tabla 2. Complejidad teórica, estabilidad y uso de memoria (n elementos, d = 8 dígitos, b = 10 (base), k = número de cubetas).

Algoritmo	Mejor caso	Caso promedio	Peor caso	Estable	In-place
Insertion Sort	O(n)	O(n²)	O(n²)	Sí	Sí
Dual-Pivot QuickSort	O(n log n)	O(n log n)	O(n²)	No	Sí (pila O(log n) en promedio)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Sí	No (O(n) auxiliar)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No	Sí
Radix Sort (LSD, base 10)	O(d·(n+b))	O(d·(n+b))	O(d·(n+b))	Sí	No (O(n+b) auxiliar)
Bucket Sort	O(n+k)	O(n+k)	O(n²)	Sí (con inserción estable)	No (O(n+k) auxiliar)

Bucket Sort. Se usan k = n cubetas de igual ancho sobre [mín, máx] del arreglo. El valor v se asigna a la cubeta ⌊(v − mín) · k / (máx − mín + 1)⌋. Las cubetas se almacenan de forma contigua (conteo y suma prefija) en lugar de una lista por cubeta, y cada cubeta se ordena con Insertion Sort, como en GeeksforGeeks.

Dual-Pivot QuickSort. Los pivotes son el primer y el último elemento (si el primero es mayor se intercambian). Con una entrada ya ordenada el pivote izquierdo es el mínimo y el derecho el máximo, por lo que cada llamada solo descarta 2 elementos: la recursión llega a profundidad ≈ n/2.

2.1 Crecimiento

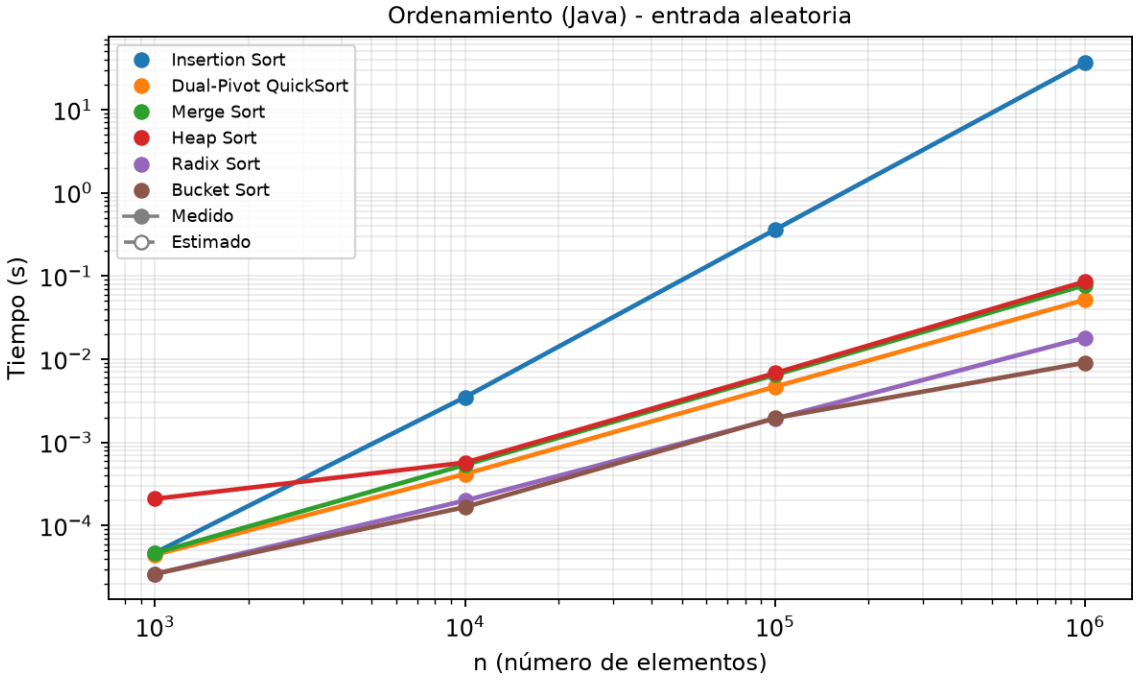


Figura 1a. Tiempo vs n (Java), entrada aleatoria, escala log-log.

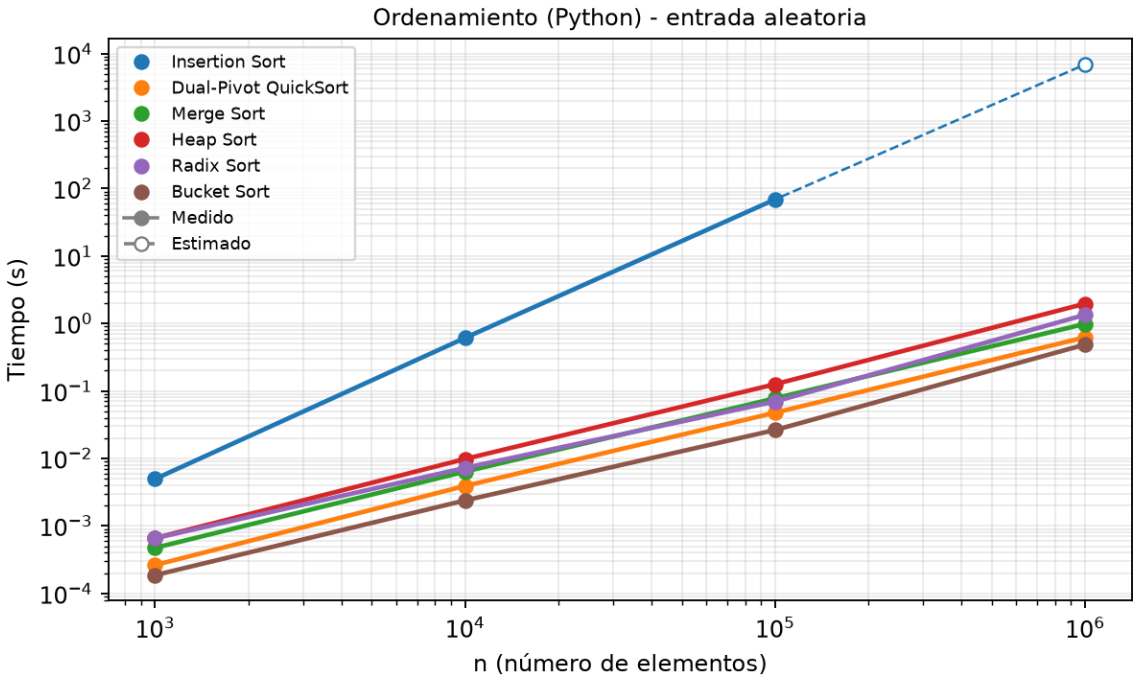


Figura 1b. Tiempo vs n (Python), entrada aleatoria, escala log-log.

Tabla 3. Factor de crecimiento del tiempo medio al multiplicar n por 10 (entrada aleatoria) frente al factor que predice la complejidad teórica. «~» indica que alguno de los extremos es estimado.

Java	**1.000 → 10.000**	**10.000 → 100.000**	**100.000 → 1.000.000**
Insertion Sort	×74.2 (teór. ×100.0)	×103.2 (teór. ×100.0)	×101.7 (teór. ×100.0)
Dual-Pivot QuickSort	×9.4 (teór. ×13.3)	×11.2 (teór. ×12.5)	×11.1 (teór. ×12.0)
Merge Sort	×11.5 (teór. ×13.3)	×12.0 (teór. ×12.5)	×12.0 (teór. ×12.0)
Heap Sort	×2.7 (teór. ×13.3)	×11.9 (teór. ×12.5)	×12.6 (teór. ×12.0)
Radix Sort	×7.6 (teór. ×10.0)	×9.6 (teór. ×10.0)	×9.4 (teór. ×10.0)
Bucket Sort	×6.4 (teór. ×10.0)	×11.7 (teór. ×10.0)	×4.6 (teór. ×10.0)

Python	**1.000 → 10.000**	**10.000 → 100.000**	**100.000 → 1.000.000**
Insertion Sort	×123.3 (teór. ×100.0)	×112.8 (teór. ×100.0)	~×100.0 (teór. ×100.0)
Dual-Pivot QuickSort	×14.9 (teór. ×13.3)	×12.2 (teór. ×12.5)	×13.1 (teór. ×12.0)
Merge Sort	×13.5 (teór. ×13.3)	×12.2 (teór. ×12.5)	×12.7 (teór. ×12.0)
Heap Sort	×14.9 (teór. ×13.3)	×12.8 (teór. ×12.5)	×15.6 (teór. ×12.0)
Radix Sort	×11.0 (teór. ×10.0)	×9.6 (teór. ×10.0)	×19.3 (teór. ×10.0)
Bucket Sort	×12.8 (teór. ×10.0)	×11.1 (teór. ×10.0)	×18.5 (teór. ×10.0)

PENDIENTE: Análisis de crecimiento (Punto 1). Redactar con las Figuras 1a/1b y la Tabla 3:

- **Para cada algoritmo: ¿cuántas veces aumenta el tiempo cada vez que n se multiplica por 10? Compararlo con el factor teórico de la Tabla 3 y explicar las diferencias (caché, JIT, recolector de basura, constantes, ruido a n pequeño).**
- **Explicar cómo se estimaron los tiempos de los casos que no terminaron (escalamiento t(n) = t(n_prev)·f(n)/f(n_prev)) y listar cuáles fueron (ver Tabla A.3).**

2.2 Efecto del orden de entrada (lenguaje compilado: Java)

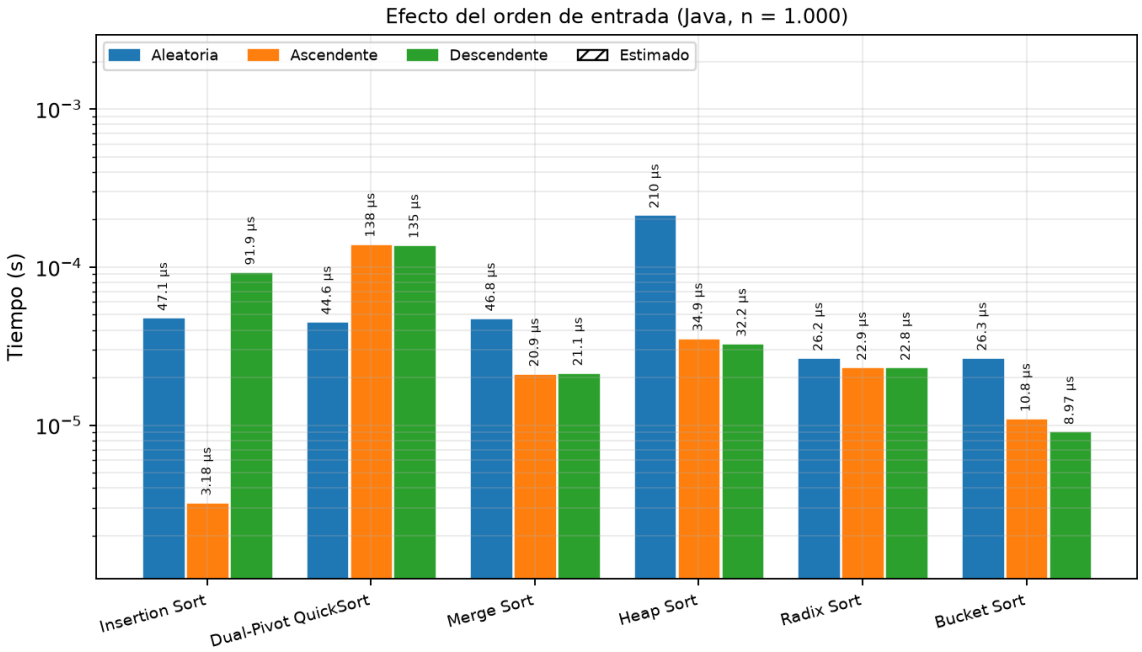


Figura 2 (n = 1.000). Tiempo por tipo de entrada, Java.

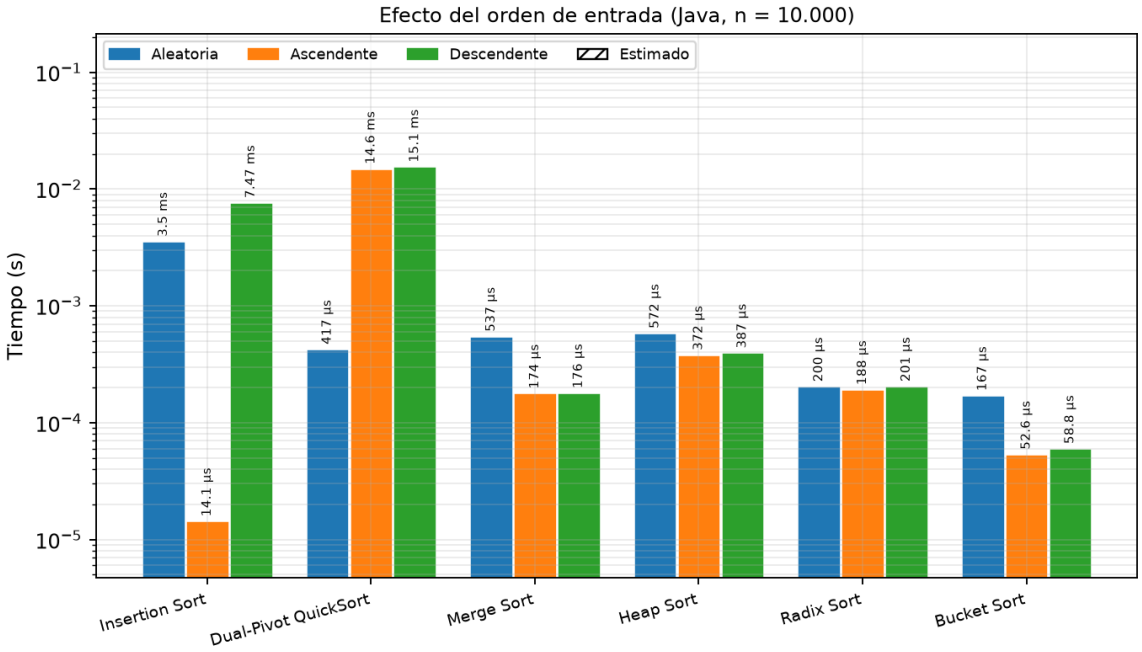


Figura 2 (n = 10.000). Tiempo por tipo de entrada, Java.

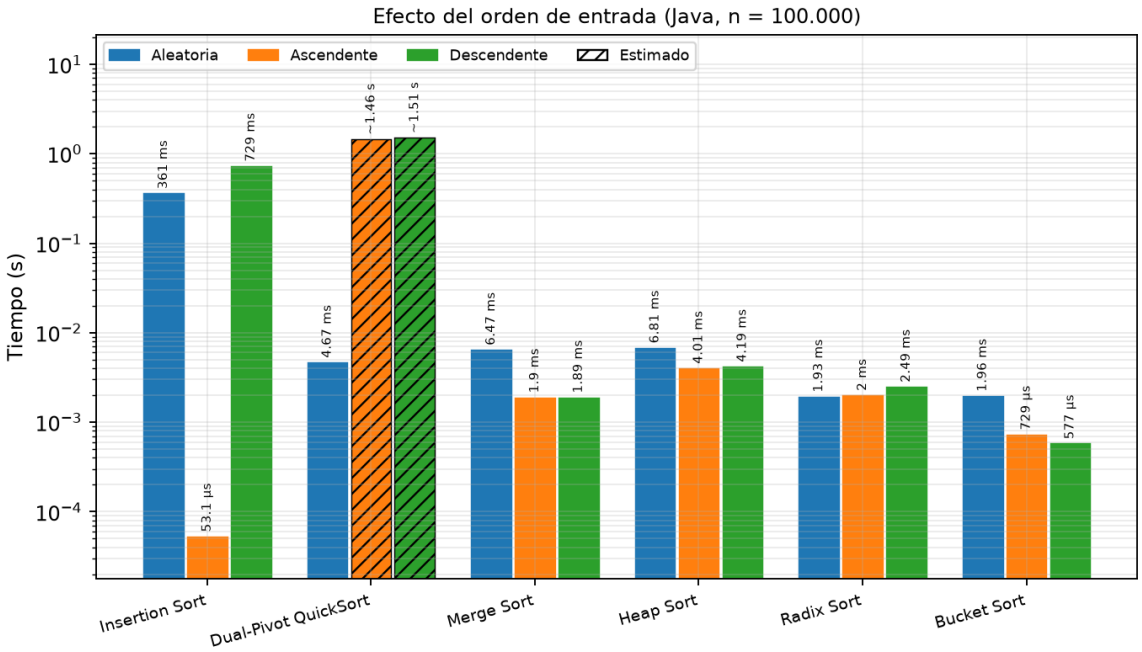


Figura 2 (n = 100.000). Tiempo por tipo de entrada, Java.

PENDIENTE: Efecto del orden de entrada (Punto 1). Redactar con las Figuras 2:

• ¿Qué algoritmos cambiaron significativamente su tiempo con la entrada ascendente o descendente y cuáles no? Explicar por qué según el funcionamiento interno de cada uno (Insertion Sort: mejor caso O(n) en ascendente y peor caso en descendente; Dual-Pivot con pivotes en los extremos; Merge/Heap/Radix/Bucket casi insensibles).

• ¿En qué caso se presentó el peor comportamiento y qué lo provocó? Mencionar el StackOverflowError / RecursionError de Dual-Pivot QuickSort si ocurrió (ver Tabla A.3) y su causa: profundidad de recursión ≈ n/2.

2.3 Compilado frente a interpretado

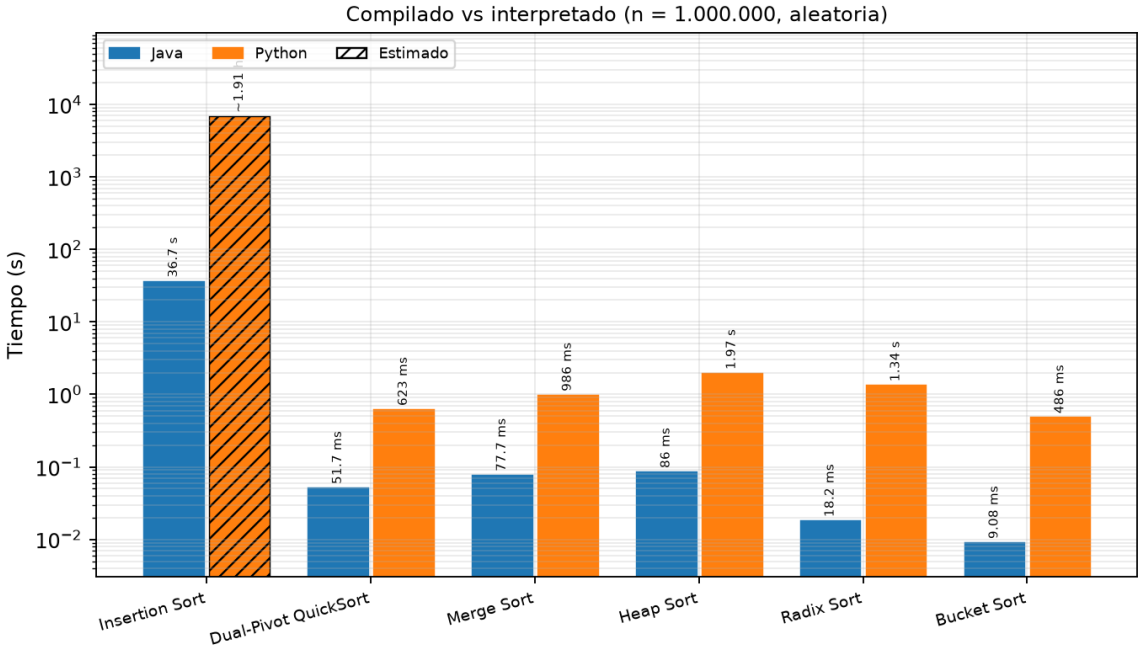


Figura 3. Java vs Python, n = 1.000.000, entrada aleatoria.

PENDIENTE: Compilado frente a interpretado, ordenamiento (Punto 1). Redactar con la Figura 3: explicar las diferencias de desempeño entre Java y Python (compilación JIT a

código máquina, tipos primitivos int[] frente a objetos int en listas de Python, costo de las llamadas a función en Python, recursión, bucles interpretados) e indicar en qué algoritmos la brecha es mayor o menor y por qué.

3. Punto 2 - Algoritmos de búsqueda

Tabla 4. Complejidad teórica (sobre un arreglo ordenado de n elementos).

Algoritmo	Mejor caso	Peor caso / promedio	Supuesto
Lineal	O(1)	O(n)	Ninguno (no requiere orden)
Binaria	O(1)	$O(\log_2 n)$	Arreglo ordenado
Ternaria	O(1)	$O(\log_3 n)$ iteraciones (≈2 comparaciones por iteración)	Arreglo ordenado
Por saltos	O(1)	$O(\sqrt{n})$	Arreglo ordenado
Interpolación	O(1)	Promedio O(log log n) con datos uniformes; peor caso O(n)	Ordenado y distribuido uniformemente
Exponencial	O(1)	O(log i), i = posición de la clave	Arreglo ordenado

Los arreglos de entrada son los ordenados ascendentemente del Punto 1. Cada medición busca el conjunto de 1.000 claves (500 presentes y 500 ausentes), mide el tiempo total y reporta el promedio por búsqueda. Para la posición de la clave, se repite 1.000 veces la búsqueda de una clave ubicada al inicio, al centro y al final del arreglo de n = 1.000.000.

3.1 Crecimiento

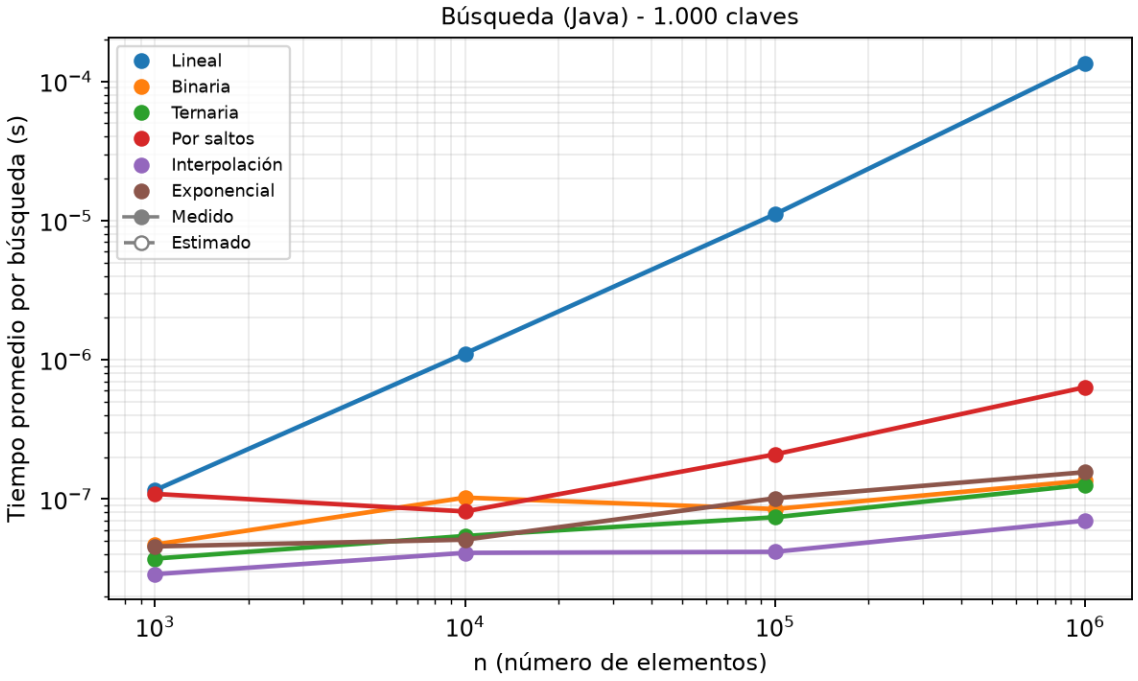


Figura 4a. Tiempo promedio por búsqueda vs n (Java), escala log-log.

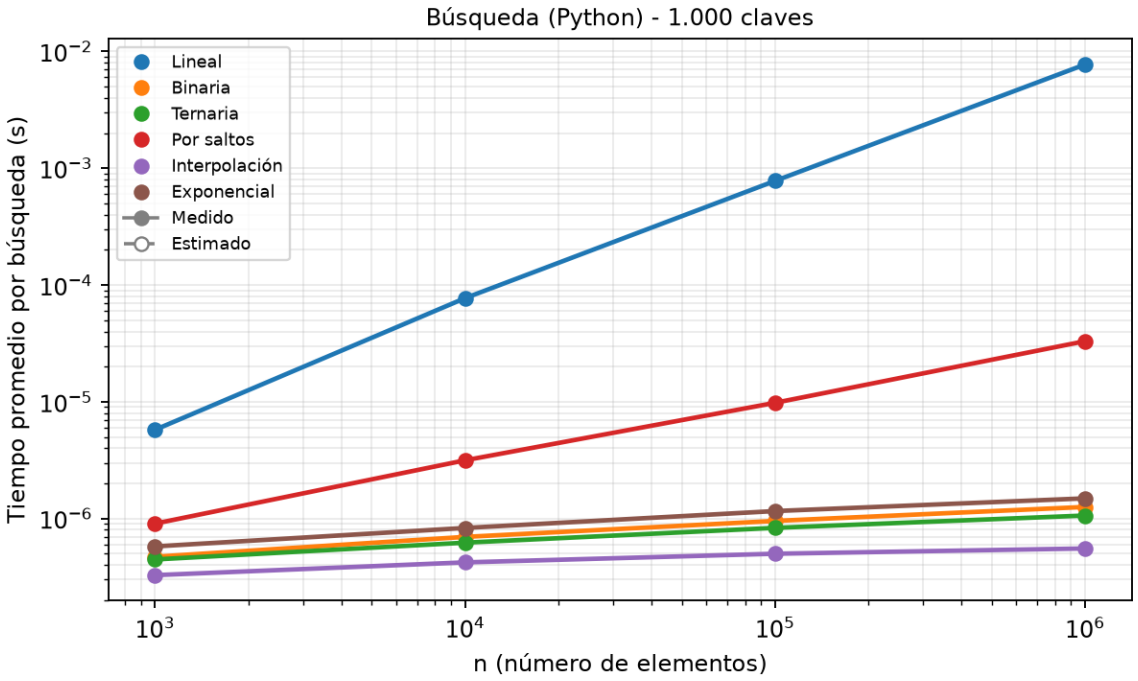


Figura 4b. Tiempo promedio por búsqueda vs n (Python), escala log-log.

PENDIENTE: Crecimiento de los algoritmos de búsqueda (Punto 2). Redactar con las Figuras 4a/4b: comparar el crecimiento observado de cada algoritmo con su complejidad teórica (O(n), O(log n), O(√n), O(log log n)), indicar cuál es más rápido y a partir de qué n se nota la diferencia, y explicar por qué la búsqueda lineal es la referencia.

3.2 Compilado frente a interpretado

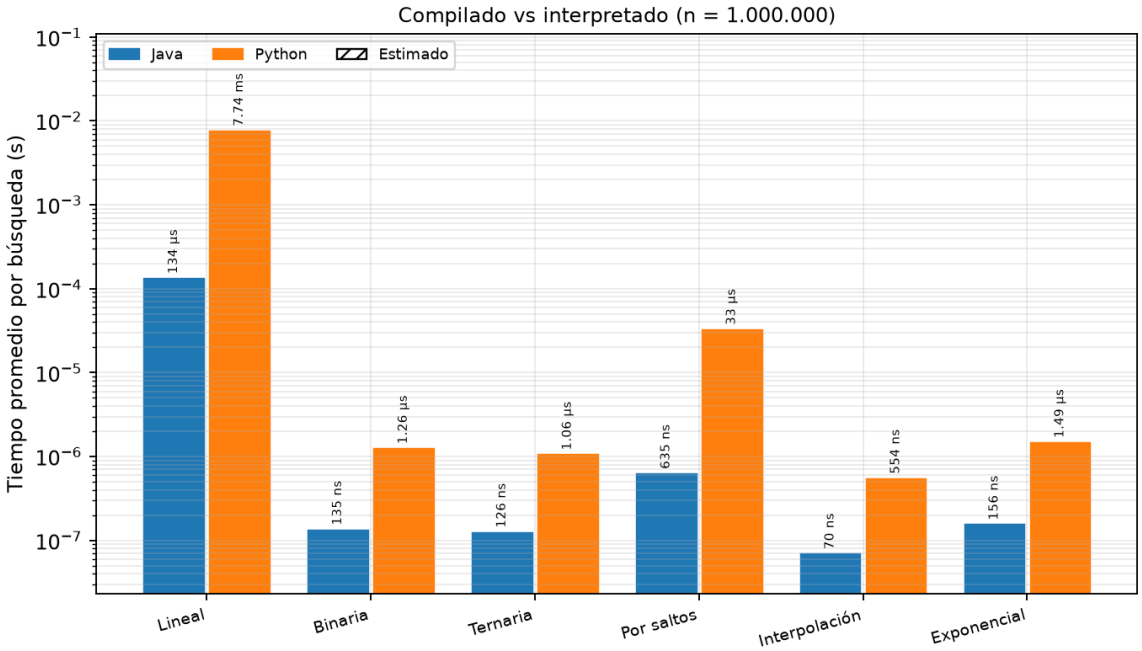


Figura 5. Java vs Python, n = 1.000.000.

PENDIENTE: Compilado frente a interpretado, búsqueda (Punto 2). Redactar con la Figura 5: explicar las diferencias entre Java y Python para cada algoritmo (la brecha debería ser mayor en los algoritmos con más iteraciones, como el lineal y por saltos, y menor en los logarítmicos, donde el tiempo por búsqueda es muy pequeño y pesa más el costo fijo de llamar a la función).

3.3 Posición de la clave

Tabla 5 (Java). Tiempo promedio por búsqueda según la posición de la clave.

Java (n = 1.000.000)	Inicio	Centro	Final
Lineal	0.933 ± 0.4 ns	72.6 ± 1.5 µs	156 ± 11 µs
Binaria	22.3 ± 1.1 ns	21.2 ± 1.9 ns	23.1 ± 0.59 ns
Ternaria	23.6 ± 0.55 ns	24.3 ± 1.3 ns	33.7 ± 15 ns
Por saltos	3.19 ± 0.18 ns	300 ± 3.6 ns	1.21 ± 0.058 µs
Interpolación	3.12 ± 0.33 ns	7.23 ± 0.2 ns	3.28 ± 0.28 ns
Exponencial	27.3 ± 20 ns	48.3 ± 14 ns	34.8 ± 4.2 ns

Tabla 5 (Python). Tiempo promedio por búsqueda según la posición de la clave.

Python (n = 1.000.000)	Inicio	Centro	Final
Lineal	66.6 ± 2.5 ns	5.12 ± 0.028 ms	10.2 ± 0.061 ms
Binaria	759 ± 10 ns	793 ± 17 ns	804 ± 8.7 ns
Ternaria	696 ± 12 ns	806 ± 13 ns	770 ± 8.1 ns
Por saltos	105 ± 1.3 ns	18.9 ± 0.1 µs	65 ± 1.9 µs
Interpolación	108 ± 4.4 ns	312 ± 3.4 ns	125 ± 6.7 ns
Exponencial	36.9 ± 2.1 ns	1.03 ± 0.012 µs	1.2 ± 0.0089 µs

PENDIENTE: Posición de la clave (Punto 2). Redactar con la Tabla 5: ¿en qué algoritmos cambia el tiempo según la posición de la clave (inicio, centro, final) y en cuáles no? Explicar por qué según el orden en que cada algoritmo revisa los elementos (lineal: recorre desde el inicio; binaria/ternaria: primer punto de comparación en el centro o a un tercio; saltos: bloques de √n desde el inicio; interpolación: estima la posición por los valores; exponencial: duplica desde el inicio).

4. Punto 3 - Análisis comparativo

4.1 Comparativos frente a no comparativos

Tabla 6. Tiempos con n = 1.000.000 (entrada aleatoria).

Algoritmo	Tipo	Java	Python
Insertion Sort	Comparativo	36.7 ± 0.07 s	~1.91 h (est.)
Dual-Pivot QuickSort	Comparativo	51.7 ± 0.35 ms	623 ± 4.2 ms
Merge Sort	Comparativo	77.7 ± 5.8 ms	986 ± 14 ms
Heap Sort	Comparativo	86 ± 1.1 ms	1.97 ± 0.11 s
Radix Sort	No comparativo	18.2 ± 0.57 ms	1.34 ± 0.097 s
Bucket Sort	No comparativo	9.08 ± 2.8 ms	486 ± 50 ms

PENDIENTE: Comparativos frente a no comparativos (Punto 3.1). Redactar con la Tabla 6:

- **Investigar el límite inferior teórico de los ordenamientos por comparaciones, Ω(n log n) (argumento del árbol de decisión: n! hojas, altura ≥ $\log_2(n!) \approx n \log_2 n$), y explicar por qué Radix Sort y Bucket Sort no están sujetos a él (no comparan elementos entre sí: usan los dígitos o el valor para ubicarlos).**
- **Comparar sus tiempos con los de los algoritmos por comparaciones y explicar qué supuesto hace cada uno (Radix: claves enteras con d dígitos fijos, costo O(d·n), crecería con números de 16 dígitos; Bucket: distribución uniforme en un rango, con valores concentrados en un rango pequeño casi todo cae en pocas cubetas y degenera a $O(n^2)$).**

4.2 ¿Ordenar o no ordenar?

Con k búsquedas sobre un arreglo desordenado de n elementos: (A) k búsquedas lineales cuestan $k \cdot t_{lineal}$; (B) ordenar una vez y luego k búsquedas binarias cuestan $T_{orden} + k \cdot t_{binaria}$. Ordenar conviene cuando $T_{orden} + k \cdot t_{binaria} < k \cdot t_{lineal}$, es decir, $k > k^* = T_{orden} / (t_{lineal} - t_{binaria})$. La búsqueda lineal de las 1.000 claves recorre el mismo número de elementos en un arreglo ordenado que en uno desordenado (no aprovecha el orden), por lo que se usa ese tiempo medido como t_{lineal}.

Tabla 7. Cálculo del umbral k* con n = 1.000.000.

Lenguaje	Mejor ordenamiento (medido)	T_orden	t_lineal	t_binaria	k*
Java	Bucket Sort	9.08 ms	134 µs	135 ns	67.7 → 68 búsquedas
Python	Bucket Sort	486 ms	7.74 ms	1.26 µs	62.8 → 63 búsquedas

Java: k* = 9.08 ms / (134 µs − 135 ns) = 67.71 ⇒ a partir de k = 68 búsquedas conviene ordenar.

Python: k* = 486 ms / (7.74 ms − 1.26 µs) = 62.83 ⇒ a partir de k = 63 búsquedas conviene ordenar.

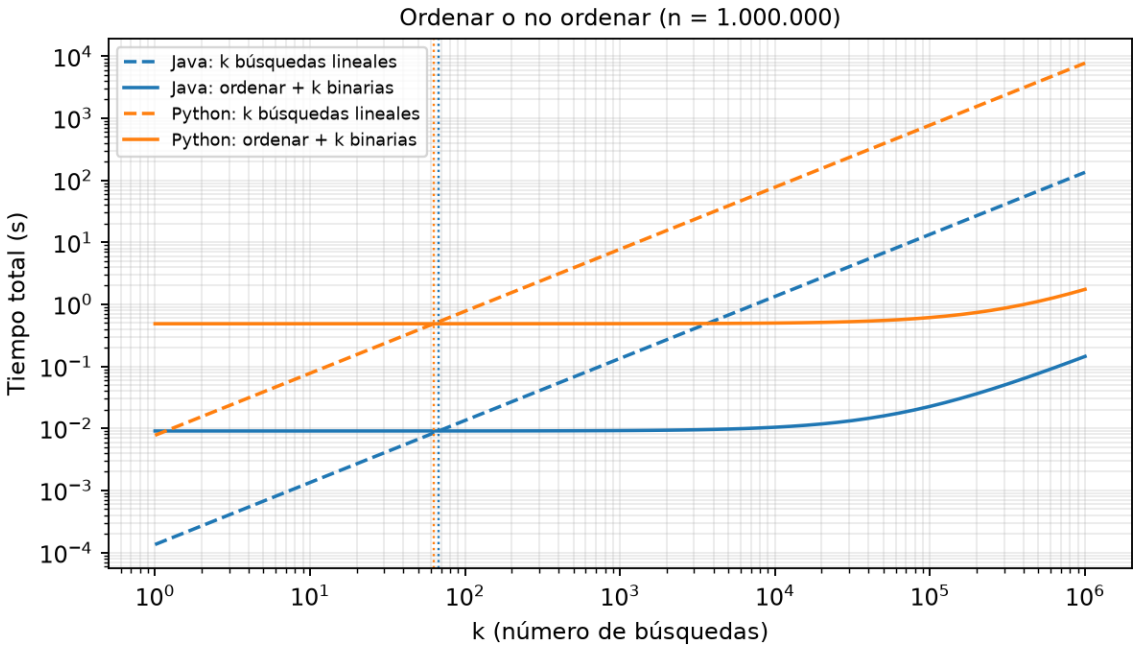


Figura 6. Costo total de cada estrategia según k (las líneas punteadas verticales marcan k).*

PENDIENTE: Interpretación de ¿Ordenar o no ordenar? (Punto 3.2). Redactar con la Tabla 7 y la Figura 6: interpretar el valor de k* para cada lenguaje (cuántas búsquedas hacen falta para que ordenar una vez y usar búsqueda binaria compense), por qué difiere entre Java y Python, y qué ocurre si k es menor o mayor que k*.

5. Uso de inteligencia artificial

Se utilizó Claude Code (Anthropic) como herramienta de apoyo en las siguientes partes del trabajo:

- **Implementación de los algoritmos:** primera versión del código en Java y Python, adaptada de los artículos de GeeksforGeeks citados en la Tabla 1, y de los programas de medición (calentamiento, repeticiones, verificación, estimación de tiempos).
- **Generación de gráficas y del documento:** scripts que leen los CSV de resultados y producen las gráficas, las tablas y este PDF.
- **Investigación y redacción:** apoyo en la consulta de las complejidades teóricas y en borradores de los análisis, que fueron revisados, corregidos y reescritos por el grupo a partir de sus propias mediciones.

Las mediciones se ejecutaron y los resultados se interpretaron por el grupo; la herramienta no generó ningún dato experimental.

(Ajustar este texto para que describa con exactitud el uso real que hizo el grupo.)

6. Conclusiones

PENDIENTE: Conclusiones generales. Resumir, con tus propias palabras y con cifras de tus mediciones, qué algoritmos resultaron mejores en cada caso, si el comportamiento coincidió con la teoría y qué sorprendió.

Anexo A. Tablas completas de resultados

Promedio ± desviación estándar muestral de 5 repeticiones. «~ ... (est.)» = valor estimado, no medido.

Tabla A.1 - Ordenamiento, Java, entrada aleatoria.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	47.1 ± 7 µs	3.5 ± 0.062 ms	361 ± 0.75 ms	36.7 ± 0.07 s
Dual-Pivot QuickSort	44.6 ± 13 µs	417 ± 70 µs	4.67 ± 0.26 ms	51.7 ± 0.35 ms
Merge Sort	46.8 ± 7.6 µs	537 ± 19 µs	6.47 ± 0.17 ms	77.7 ± 5.8 ms
Heap Sort	210 ± 3.6e+02 µs	572 ± 40 µs	6.81 ± 0.12 ms	86 ± 1.1 ms
Radix Sort	26.2 ± 4.4 µs	200 ± 5.4 µs	1.93 ± 0.1 ms	18.2 ± 0.57 ms
Bucket Sort	26.3 ± 11 µs	167 ± 64 µs	1.96 ± 0.88 ms	9.08 ± 2.8 ms

Tabla A.1 - Ordenamiento, Java, entrada ascendente.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	3.18 ± 2.7 µs	14.1 ± 11 µs	53.1 ± 0.59 µs	572 ± 77 µs
Dual-Pivot QuickSort	138 ± 3.8 µs	14.6 ± 0.12 ms	~1.46 s (est.)	~2.43 min (est.)
Merge Sort	20.9 ± 1.6 µs	174 ± 4.3 µs	1.9 ± 0.049 ms	28.8 ± 16 ms
Heap Sort	34.9 ± 2.1 µs	372 ± 8.5 µs	4.01 ± 0.13 ms	46.2 ± 3.2 ms
Radix Sort	22.9 ± 2.3 µs	188 ± 6.6 µs	2 ± 0.17 ms	18.4 ± 0.18 ms
Bucket Sort	10.8 ± 6.2 µs	52.6 ± 5.1 µs	729 ± 3e+02 µs	5.8 ± 0.91 ms

Tabla A.1 - Ordenamiento, Java, entrada descendente.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	91.9 ± 2.8 µs	7.47 ± 0.12 ms	729 ± 7.3 ms	1.23 ± 0.00037 min
Dual-Pivot QuickSort	135 ± 1.6 µs	15.1 ± 0.44 ms	~1.51 s (est.)	~2.52 min (est.)
Merge Sort	21.1 ± 2.5 µs	176 ± 3.1 µs	1.89 ± 0.037 ms	21 ± 0.19 ms
Heap Sort	32.2 ± 2.9 µs	387 ± 9 µs	4.19 ± 0.079 ms	45.4 ± 0.16 ms
Radix Sort	22.8 ± 1.5 µs	201 ± 10 µs	2.49 ± 0.42 ms	17.9 ± 0.22 ms
Bucket Sort	8.97 ± 1.4 µs	58.8 ± 6.9 µs	577 ± 26 µs	6.04 ± 0.12 ms

Tabla A.1 - Ordenamiento, Python, entrada aleatoria.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	4.95 ± 0.033 ms	611 ± 12 ms	1.15 ± 0.076 min	~1.91 h (est.)
Dual-Pivot QuickSort	263 ± 18 µs	3.91 ± 0.051 ms	47.5 ± 0.15 ms	623 ± 4.2 ms
Merge Sort	472 ± 16 µs	6.36 ± 0.038 ms	77.6 ± 0.27 ms	986 ± 14 ms
Heap Sort	658 ± 15 µs	9.81 ± 0.15 ms	126 ± 0.62 ms	1.97 ± 0.11 s
Radix Sort	655 ± 1e+02 µs	7.23 ± 0.49 ms	69.6 ± 0.51 ms	1.34 ± 0.097 s
Bucket Sort	186 ± 13 µs	2.38 ± 0.063 ms	26.3 ± 1.2 ms	486 ± 50 ms

Tabla A.1 - Ordenamiento, Python, entrada ascendente.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	30.9 ± 1 µs	300 ± 4.1 µs	3.22 ± 0.21 ms	30 ± 0.16 ms
Dual-Pivot QuickSort	4.48 ± 0.032 ms	~448 ms (est.)	~44.8 s (est.)	~1.24 h (est.)
Merge Sort	367 ± 3 µs	4.97 ± 0.022 ms	58.8 ± 0.068 ms	678 ± 6.1 ms
Heap Sort	670 ± 10 µs	9.45 ± 0.082 ms	121 ± 0.91 ms	1.46 ± 0.012 s
Radix Sort	635 ± 28 µs	7 ± 0.1 ms	68.3 ± 0.96 ms	924 ± 30 ms
Bucket Sort	174 ± 8.2 µs	2.17 ± 0.18 ms	21.3 ± 0.24 ms	224 ± 6.6 ms

Tabla A.1 - Ordenamiento, Python, entrada descendente.

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Insertion Sort	9.82 ± 0.17 ms	1.16 ± 0.013 s	2.07 ± 0.058 min	~3.45 h (est.)
Dual-Pivot QuickSort	4.58 ± 0.032 ms	~458 ms (est.)	~45.8 s (est.)	~1.27 h (est.)
Merge Sort	360 ± 3 µs	4.87 ± 0.019 ms	57.9 ± 0.18 ms	665 ± 2.1 ms
Heap Sort	589 ± 6.8 µs	8.52 ± 0.03 ms	111 ± 0.45 ms	1.41 ± 0.029 s

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Radix Sort	611 ± 4.7 µs	6.88 ± 0.064 ms	68.3 ± 1.1 ms	1.07 ± 0.13 s
Bucket Sort	183 ± 2.1 µs	2.18 ± 0.023 ms	22.2 ± 0.2 ms	262 ± 36 ms

Tabla A.2 - Búsqueda, Java, tiempo por búsqueda (1.000 claves).

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Lineal	116 ± 3.3 ns	1.11 ± 0.028 µs	11.1 ± 0.32 µs	134 ± 20 µs
Binaria	47.1 ± 6.1 ns	102 ± 90 ns	84.7 ± 13 ns	135 ± 50 ns
Ternaria	37.3 ± 12 ns	54.3 ± 11 ns	74 ± 18 ns	126 ± 54 ns
Por saltos	109 ± 59 ns	81.4 ± 4.6 ns	209 ± 11 ns	635 ± 46 ns
Interpolación	28.9 ± 2.2 ns	41.1 ± 9.6 ns	41.8 ± 15 ns	70 ± 42 ns
Exponencial	45.6 ± 7.7 ns	51 ± 8.2 ns	101 ± 14 ns	156 ± 52 ns

Tabla A.2 - Búsqueda, Python, tiempo por búsqueda (1.000 claves).

Algoritmo	n = 1.000	n = 10.000	n = 100.000	n = 1.000.000
Lineal	5.76 ± 0.23 µs	77.2 ± 2.6 µs	779 ± 8.3 µs	7.74 ± 0.099 ms
Binaria	471 ± 22 ns	696 ± 52 ns	954 ± 2e+02 ns	1.26 ± 0.39 µs
Ternaria	446 ± 17 ns	621 ± 24 ns	832 ± 93 ns	1.06 ± 0.26 µs
Por saltos	905 ± 12 ns	3.15 ± 0.017 µs	9.8 ± 0.072 µs	33 ± 0.74 µs
Interpolación	328 ± 9.8 ns	421 ± 18 ns	500 ± 96 ns	554 ± 1.3e+02 ns
Exponencial	577 ± 13 ns	831 ± 23 ns	1.16 ± 0.13 µs	1.49 ± 0.28 µs

Tabla A.3 - Registro de valores estimados y fallos.

Lenguaje	Algoritmo	Escenario	n	Estado	Nota
Java	DualPivotQuickSort	asc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Java	DualPivotQuickSort	desc	100.000	ERROR_StackOverflowError	StackOverflowError at n=100000; scaled from n=10000
Java	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=10000
Python	DualPivotQuickSort	asc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	asc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	asc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	10.000	ERROR_RecursionError	RecursionError at n=10000 (limit 1000); scaled from n=1000
Python	DualPivotQuickSort	desc	100.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	DualPivotQuickSort	desc	1.000.000	ESTIMATED	same failure expected (recursion depth grows with n); scaled from n=1000
Python	InsertionSort	desc	1.000.000	ESTIMATED	predicted 12413s > limit 300s; scaled from n=100000
Python	InsertionSort	random	1.000.000	ESTIMATED	predicted 6890s > limit 300s; scaled from n=100000

Reproducción. Código fuente, datos (o el programa que los genera con semilla fija) y archivos de resultados: https://github.com/USUARIO/REPOSITORIO.